- An account tries to vote for a proposal number that doesn’t exist. This transaction is rejected by the condition in the `vote()` function.

  This test again concerns the `vote()` function. Close the current deployment by clicking the X button in the top-right corner of the user interface. Redeploy the contract by clicking the Deploy button and entering 3 as a parameter to the constructor. Register an account as a voter, and change `state` to 2 or `Phase.vote`. Now enter a number (>= 3) in the `vote()` function’s parameter box, and click `vote`. Because the valid proposal numbers are 0, 1, and 2, this should error out as validated by the condition in the `vote()` function: `toProposal >= proposals.length`.

These examples should give you an idea about testing the smart contract. In this case, you are testing it manually in the interface provided by the Remix IDE, and you can review the errors in the console. In chapter 10, after you’ve developed the entire application stack, you’ll learn to write test scripts, thus automating the manual test process.

3.4 Using modifiers, require(), and revert()

You’ve learned how to define a rule by using the modifier feature in Solidity. What if you have more than one rule for executing a function? You can apply a series of rules (access modifiers) to a function invocation. What if a condition is to be checked during or after the execution of the statements within a function? In this case, you can use a `require()` clause that reverts the function if the condition specified within it fails. The modifier